

Neural Networks

Academic Year 1447 Term 1 Dr. Jomana Bashatah
Slides adapted from Casten Lange

Learning Outcomes

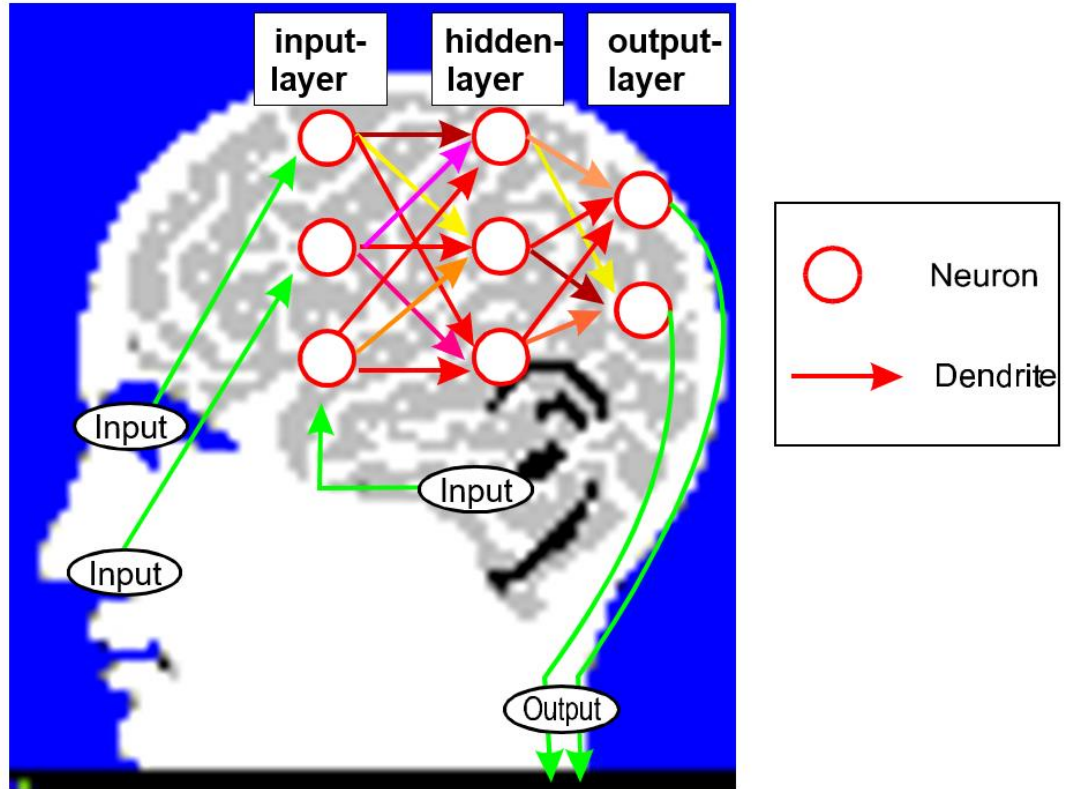
- How to work with a graphical representation of a Neural Network
- How to transform the graphical representation into a Neural Network prediction function
- How to use an Optimizer in a Neural Network to change the network's parameters to step-wise improve the approximation quality
- Why Neural Networks have outstanding approximation qualities that allow them to approximate any continuous function with any degree of accuracy
- How and why the outstanding approximation quality of Neural Networks makes them prone to overfitting
- How to work with the R nnet package and with PyTorch to design and run a Neural Network
- Why a Neural Network with ReLU (Rectified Linear Unit) activation functions has the same outstanding approximation properties as a Neural Network with classic Sigmoid activation functions
- Why ReLU functions mitigate the possible inability of the Optimizer to change the values for the β parameters
- How to use PyTorch in an interactive project to estimate the prices of more than 50,000 diamonds based on four common predictors used in the appraisal industry (Carat, Clarity, Cut, and Color)

An Introduction

- Deep Learning (DL) is an Artificial Intelligence (AI) methodology. It performs tasks that we all believed just a few years ago, could only be performed by humans.
- DL Models are used in many different applications
 - Image Recognition: Using DL to analyze medical images to improve cancer diagnose
 - Natural Language Processing (NLP): Amazon's Alexa uses DL to understand human language
 - Conversational Generative Pre-Trained Transformers (Chatbots): Use DL to compose written text
- All these applications are based on Deep Learning
- Deep Learning uses Neural Networks as an integral part
 - There would be no Deep Learning without Neural Networks

The Early Days

In the early days of artificial neural networks, data scientists tried to mimic the human brain through computer models.



The Early Days

- The human brain processes information as electric currents through neurons, and then passes it onto other neurons connected by dendrites in a complex network with endless layers of neurons.
 - Similarly, a Neural Network contains layers of interconnected nodes—like neurons in the human brain. Each of these nodes, or artificial neurons, performs a mathematical operation on the data it receives, and then passes the results on to other neurons in the network.
 - Neural Network research today focuses on improving predictive quality, rather than attempting to resemble the human brain

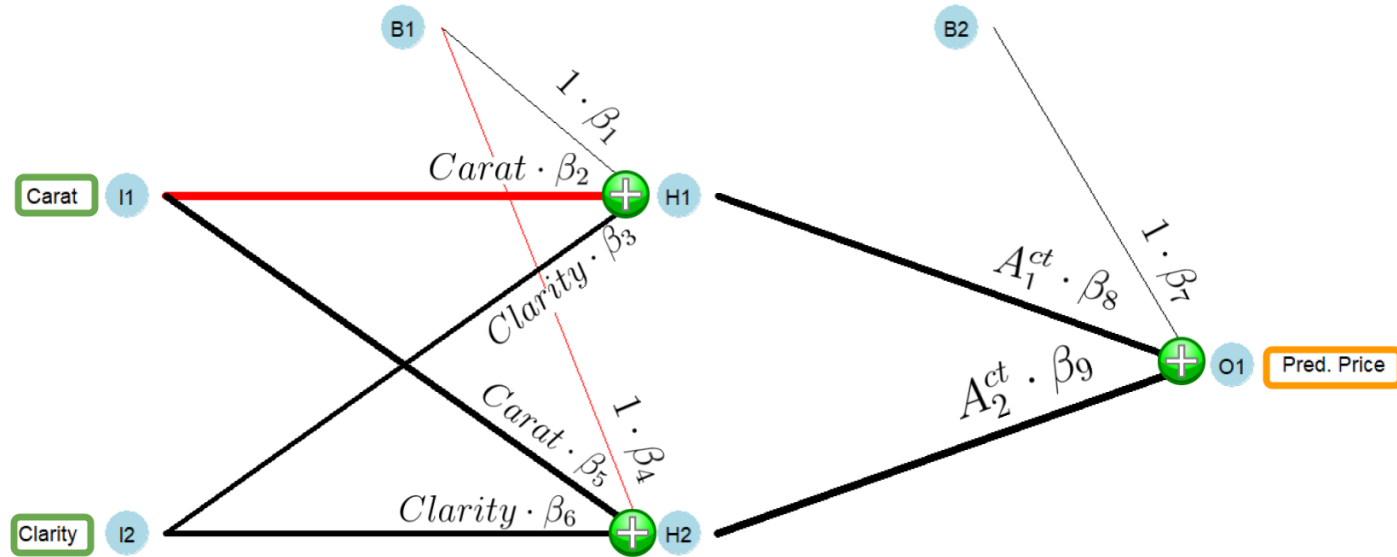
Types of Neural Networks

- The word *Deep* in DL: Number of layers in the Neural Network structure
- In general, Deep Learning algorithms are extremely complex with many layers and a very high number of parameters (*sometimes billions or trillions*)
- Examples of Deep Neural Networks:
 - [Convolutional Neural Networks \(CNN\)](#)
 - Used for image recognition and classification
 - Recurrent Neural Networks: used for sequence processing
 - Long Short Term Memory recurrent networks: a more advanced type of RNN for tasks that require sequence processing, such as speech recognition and sentiment analysis.
 - Generative Adversarial Networks
 - for semantic image editing, style transfer, image synthesis.
- Due to complexity and demanding computational resources, Deep Neural Networks are beyond the scope of our course
 - We will learn the basic underlying concepts of Deep Neural Networks using a simpler Neural Networks type called **Multi-Layer Perceptrons (MLP)** neural networks

Multi-Layer Perceptrons (MLP) Neural Network

- Among the first Neural Networks to be developed. Their principles can be applied to more complex and sophisticated Neural Networks
- NNs consist of *one or more* layers, each layer contains *one or more* neurons
 - **Input Layer:** with one or more input neurons.
 - **Hidden Layer(s)** one or more hidden layers with one or more hidden neurons.
 - **Output Layer:** with one or more output neurons.
- The MLP Neural Network is said to be **Fully connected**
 - Each neuron *in each of the layers* can pass information to each neuron in the following layer.
- NNs can be used for classification & regression.
 - We will be applying a classification for our in class example

Example for an MLP Neural Network with One Hidden Layer



The Data

We will estimate diamond prices based on their physical properties and use the well-known `diamonds` dataset automatically loaded together with `tidymodels`:

The dataset contains information on 53,940 diamonds

```
library(tidymodels); library(janitor)
str(diamonds)
tibble [53,940 × 10] (S3: tbl_df/tbl/data.frame)
 $ carat   : num [1:53940] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
 $ cut     : Ord.factor w/ 5 levels "Fair"<"Good"<...: 5 4 2 4 2 3 3 3 1 3 ...
 $ color   : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<...: 2 2 2 6 7 7 6 5 2 5 ...
 $ clarity: Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<...: 2 3 5 4 2 6 7 3 4 5 ...
 $ depth   : num [1:53940] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
 $ table   : num [1:53940] 55 61 65 58 58 57 57 55 61 61 ...
 $ price   : int  [1:53940] 326 326 327 334 335 336 336 337 337 338 ...
 $ x       : num [1:53940] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
 $ y       : num [1:53940] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
 $ z       : num [1:53940] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...
```

See: [Source: diamonds dataset:](#)

Domain Knowledge: The Four C s to Appraise a Diamond

1. *Cut*: Refers to the facets, symmetry, and reflective qualities of a diamond. The cut of a diamond is directly related to its overall sparkle and beauty.
2. *Color*: Refers to the natural color or lack of color visible within a diamond. The closer a diamond is to “colorless,” the higher its value.
3. *Clarity*: Is the visibility of natural microscopic inclusions and imperfections within a diamond. Diamonds with little to no inclusions are considered particularly rare and highly valued.
4. *Carat*: Is the unit of measurement used to describe the weight of a diamond. It is often the most visually apparent factor when comparing diamonds.

See: [Source: Brilliant Earth](#)

Data Engineering

We start with a very basic model with 2 predictors for *Price*:

- *Carat* (the weight of the diamond in metric grams),
- *Clarity* (eight categories with 8 being the best).

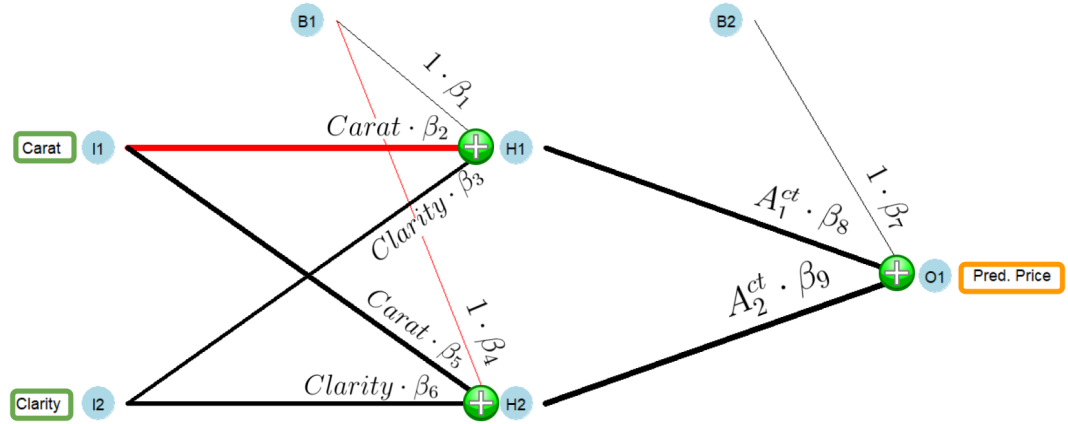
To later increase training speed, we use only 10,000 observations.

```
set.seed(777)
DataDiamonds=diamonds %>%
  clean_names("upper_camel") %>%
  select(Price, Carat, Clarity) %>%
  mutate(Clarity=as.integer(Clarity)) %>%
  sample_n(10000)

set.seed(777)
Split70=DataDiamonds %>%
  initial_split(prop = 0.7, strata = Price, breaks = 5)
DataTrain=training(Split70)
DataTest=testing(Split70)
print(DataTrain)
# A tibble: 6,999 × 3
   Price Carat Clarity
  <int> <dbl> <int>
1   506  0.3     3
2   628  0.28    6
3   753  0.3     7
4   766  0.3     6
5   552  0.35    5
6   743  0.33    5
7   698  0.31    4
8   526  0.3     4
9   675  0.3     5
10  544  0.31    4
# i 6,989 more rows
```

Use a Trained Neural Network (β s are known) to Predict

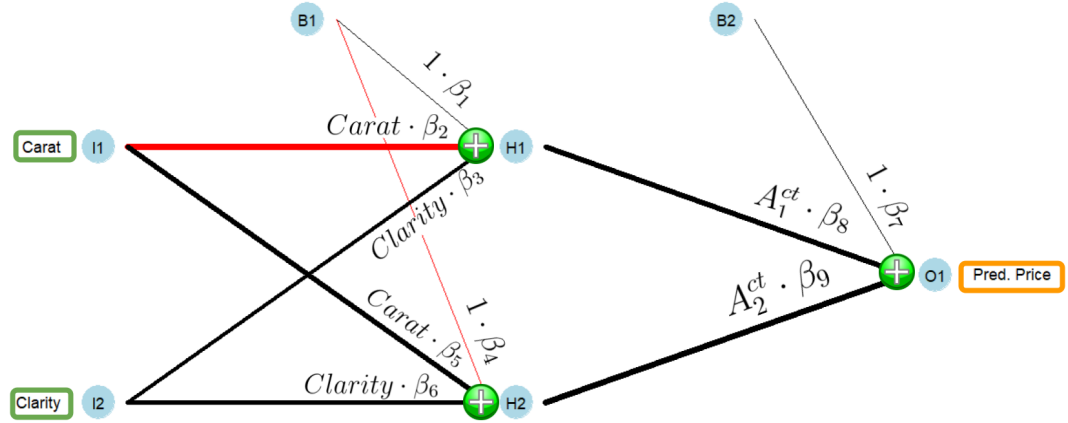
Effective Inputs to Hidden Neurons:



$$I_1^{np\,eff} = \beta_1 + \beta_2 Carat + \beta_3 Carity$$

Use a Trained Neural Network (β s are known) to Predict

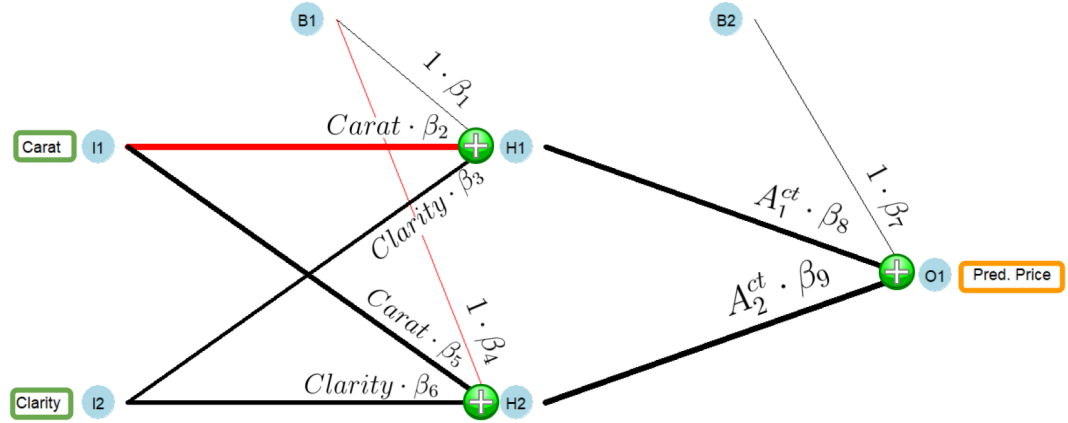
Calculate Activity in Hidden Neurons with Logistic Function



$$A_1^{ct} = \frac{1}{1 + e^{-I_1^{np\, eff}}}$$

Use a Trained Neural Network (β s are known) to Predict

Calculate Prediction from Activities in Hidden Neurons:



$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

Prediction of the Neural Network

$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

A neural network can be transformed into a prediction equation that depends only on the β s and the values of the predictor variables!

We will show this in more detail on the following slides.

Transformation From Neural Network to Prediction Equation

$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

- A_1^{ct} and A_2^{ct} depend on $I_1^{np\ eff}$ and $I_2^{np\ eff}$ (and the β s)
- $I_1^{np\ eff}$ and $I_2^{np\ eff}$ depend on the values of predictor variables *Carat* and *Clarity* (and the β s)
- Consequently, prediction depends only on the values of predictor variables and the β s!

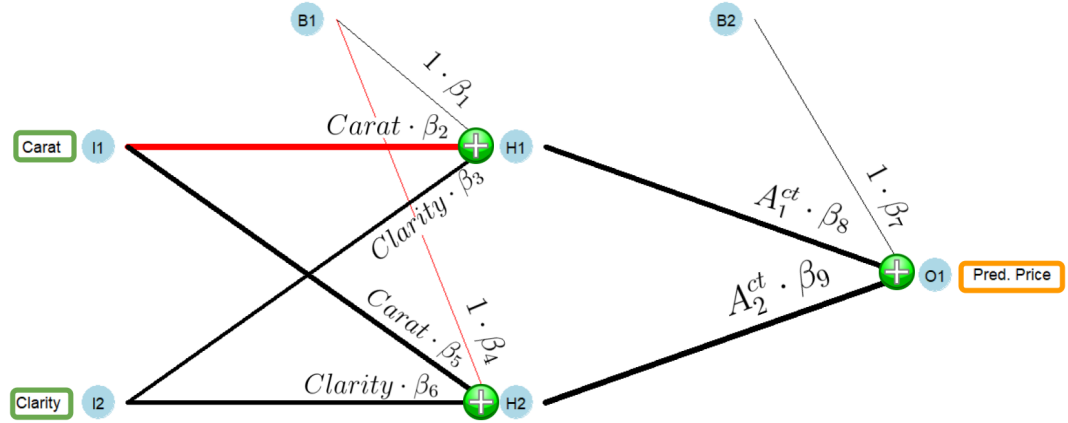
Transformation From Neural Network to Prediction Equation

To show the transformation, we move backwards from right to left through the neural network.

$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

Transformation From Neural Network to Prediction Equation

Inside the Hidden Neurons:



$$A_1^{ct} = \frac{1}{1 + e^{-I_1^{np\, eff}}}$$

Transformation From Neural Network to Prediction Equation

Inside the Hidden Neurons

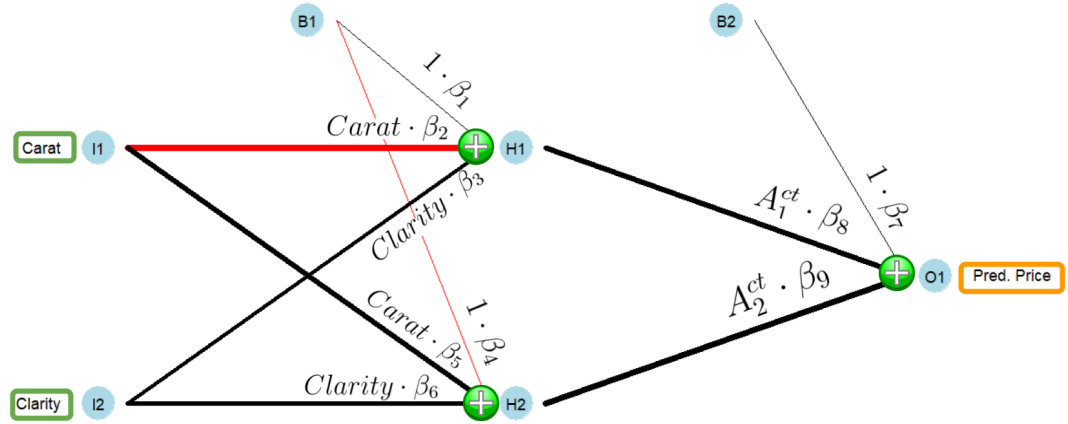
$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

...

$$\hat{P}_i = \beta_7 + \frac{\overbrace{A_1^{ct}}^1}{1 + e^{-I_1^{np\,eff}}} \cdot \beta_8 + \frac{\overbrace{A_2^{ct}}^1}{1 + e^{-I_2^{np\,eff}}} \cdot \beta_9$$

Transformation From Neural Network to Prediction Equation

Between the Input and the Hidden Layer:



$$I_1^{np\,eff} = \beta_1 + \beta_2 Carat + \beta_3 Carity$$

Transformation From Neural Network to Prediction Equation

Between the Input and the Hidden Layer:

$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$
$$\hat{P}_i = \beta_7 + \frac{\overbrace{A_1^{ct}}^1}{1 + e^{-I_1^{np\,eff}}} \cdot \beta_8 + \frac{\overbrace{A_2^{ct}}^1}{1 + e^{-I_2^{np\,eff}}} \cdot \beta_9$$

...

If we know the β s We Can Generate Predictions!

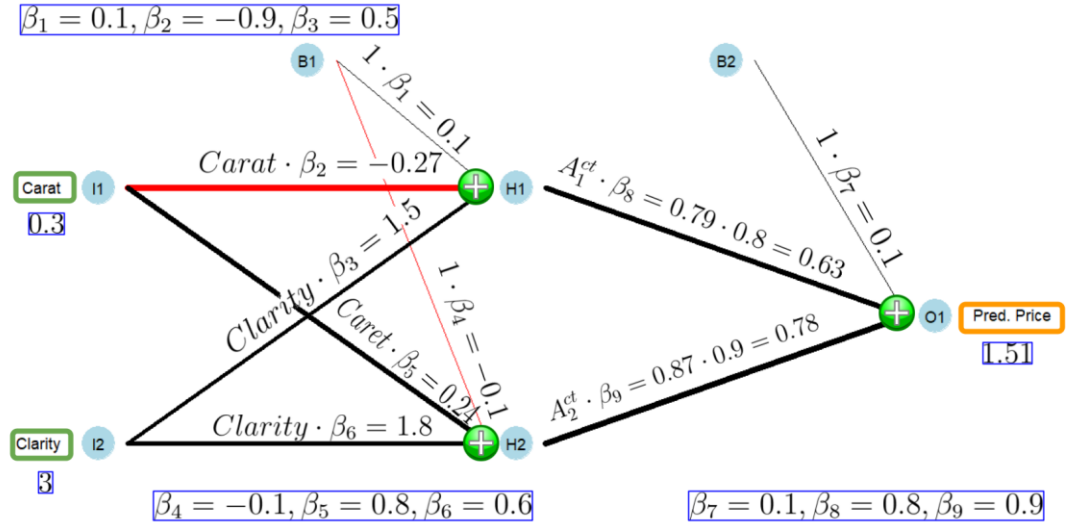
- initial β s are chosen at random.
- optimal β s are found with the *optimizer*.

Predicting the First Observation of the Training Data

Predictor Variables' Values: *Carat* = 0.3 and *Clarity* = 3

Predicting the First Observation of the Training Data

Effective Inputs: $Carat = 0.3$ and $Clarity = 3$



Predicting the First Observation of the Training Data

Effective Input 1: $Carat = 0.3$ and $Clarity = 3$

$$\beta_1 = 0.1, \beta_2 = -0.9, \beta_3 = 0.5$$

$$I_1^{np\ eff} = \beta_1 + \beta_2 Carat + \beta_3 Clarity$$

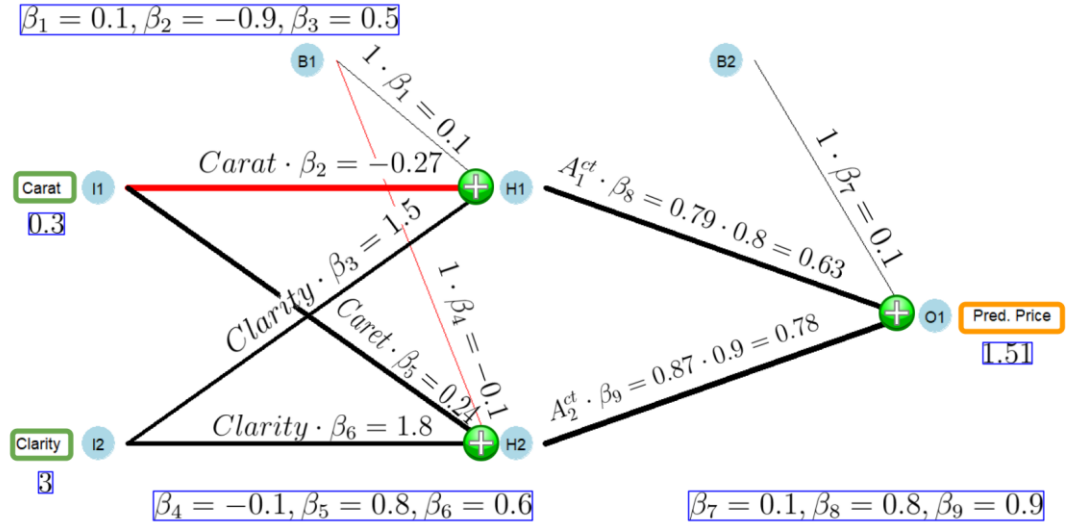
...

$$I_1^{np\ eff} = 1 \cdot 0.1 + 0.3 \cdot (-0.9) + 3 \cdot 0.5 = 1.33$$

$$1 \cdot \overset{\smile}{\beta}_1=0.1 \quad Carat \cdot \overset{\smile}{\beta}_2=-0.27 \quad Clarity \cdot \overset{\smile}{\beta}_2=First5$$

Predicting the First Observation of the Training Data

Effective Input 2: $Carat = 0.3$ and $Clarity = 3$



Predicting the First Observation of the Training Data

Effective Input 2: *Carat* = 0.3 and *Clarity* = 3

$$\beta_4 = -0.1, \beta_5 = 0.8, \beta_6 = 0.6$$

$$I_2^{np\ eff} = \beta_4 + \beta_5 Carat + \beta_6 Clarity$$

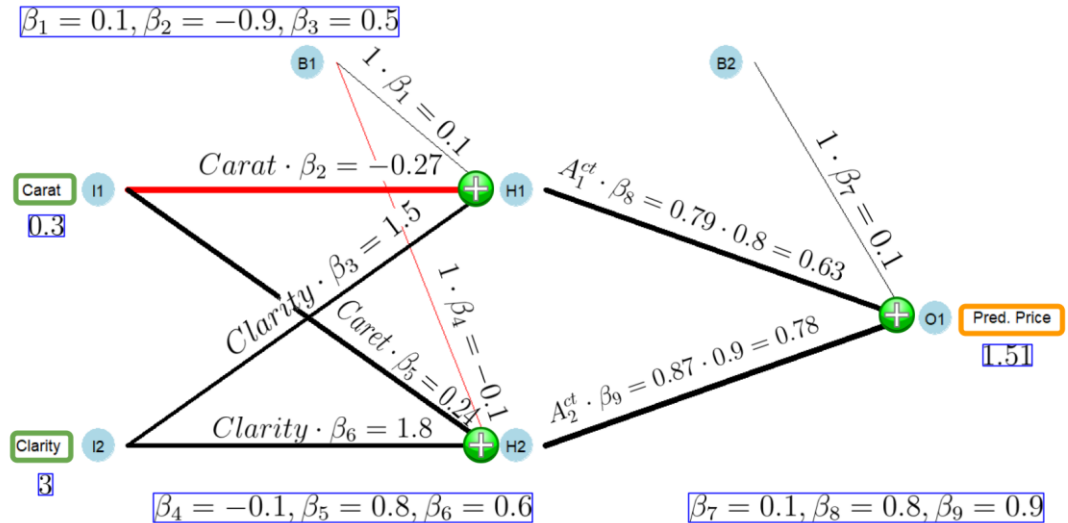
...

$$I_2^{np\ eff} = 1 \cdot (-0.1) + 0.3 \cdot 0.8 + 3 \cdot 0.6 = 1.94$$

$$1 \cdot \beta_4 \overset{\curvearrowright}{=} -0.1 \quad Carat \cdot \beta_5 \overset{\curvearrowright}{=} 0.24 \quad Clarity \cdot \beta_6 \overset{\curvearrowright}{=} 1.8$$

Predicting the First Observation of the Training Data

Hidden Neurons' Activity: $I_1^{np\ eff} = 1.33$ $I_2^{np\ eff} = 1.94$



Predicting the First Observation of the Training Data

Hidden Neurons' Activity: $I_1^{np\ eff} = 1.33$ and $I_2^{np\ eff} = 1.94$

$$A_1^{ct} = \frac{1}{1 + e^{-I_1^{np\ eff}}}$$

...

$$A_1^{ct} = \frac{1}{1 + e^{-1.33}} = 0.79$$

...

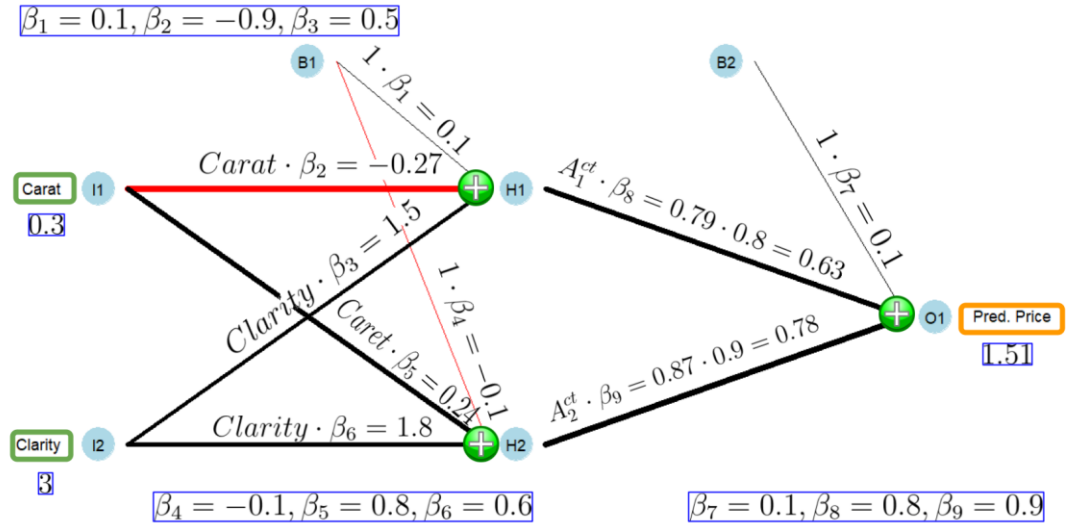
$$A_2^{ct} = \frac{1}{1 + e^{-I_2^{np\ eff}}}$$

...

$$A_2^{ct} = \frac{1}{1 + e^{-1.94}} = 0.87$$

Predicting the First Observation of the Training Data

Prediction: $\beta_7 = 0.1, \beta_8 = 0.8, \beta_9 = 0.9, A_1^{ct} = 0.79$ and $A_2^{ct} = 0.87$



Predicting the First Observation of the Training Data

Prediction: $\beta_7 = 0.1$, $\beta_8 = 0.8$, $\beta_9 = 0.9$, $A_1^{ct} = 0.79$ and $A_2^{ct} = 0.87$

$$\hat{P} = \beta_7 + \beta_8 A_1^{ct} + \beta_9 A_2^{ct}$$

...

$$\hat{P} = 0.1 + 0.8 \cdot 0.79 + 0.9 \cdot 0.87 = 1.51$$

The predicted price for a 0.3 g diamond with a clarity level of three is \$1.51. **\$1.51 for a diamond???**

Summary

- We can make prediction with the neural network if we know the values for the β s. We do know the β s because
 - they are randomly chosen at the beginning, or
 - they are adjusted by the *Optimizer*.
- when β 's are randomly chosen the predictions are usually bad, but they can be improved by the *Optimizer*.

...

This raises the question:

How does the *Optimizer* gradually change the β s to improve the prediction quality of the neural network?

Steepest Gradient Descent

$$MSE = \frac{\sum_{i=1}^N (\hat{P}_i - P_i)^2}{N}$$

Steepest Gradient Descent

$$MSE = \frac{\sum_{i=1}^N (\hat{P}_i - P_i)^2}{N}$$

Steepest Gradient Descent

- Initially β s are chosen randomly.
- Optimizer adjusts β s incrementally (iteration by iteration; the iterations are called **epochs**)
- Each epoch:
 - Find if individual β needs to be increased or decreased.
 - Increase β_i and see if MSE increases or not.
 - Decrease β_i and see if MSE increases or not.
 - Reset β_i and note if β_i needs to be increased or decreased.
 - Repeat for all β s
 - In/Decrease β 's proportional to change of MSE caused — multiply by learning rate (e.g., 0.01) to keep change small.
- run process for several hundreds or thousands epochs.

Example: Approximation Properties of Neural Networks

Let us run an example to see how well a Neural Network can approximate.

In the example we will z-normalize the predictors.

Buils a Simplified Neural Network in R

Start with loading the data

```
library(tidymodels); library(janitor)
set.seed(777)
DataDiamonds=diamonds |>
clean_names("upper_camel") |>
select(Price, Carat, Clarity) |>
mutate(Clarity=as.numeric(Clarity)) |>
sample_n(500)
Split005= initial_split(DataDiamonds,
prop=0.05)
DataTrain=training(Split005)
DataTest=testing(Split005)
```

Buils a Simplified Neural Network in R

Next, define the recipe()

```
RecipeDiamonds=recipe(Price~.,  
data=DataTrain) |>  
step_normalize(all_predictors())
```

And... Create the Model Design

```
ModelDesignNN= mlp(hidden_units=50,  
epochs=10000, penalty=0) |>  
set_engine("nnet") |>  
set_mode("regression")
```

Buils a Simplified Neural Network in R

Next, fit a workflow()

```
WFModelNN=workflow() |>  
  add_model(ModelDesignNN) |>  
  add_recipe(RecipeDiamonds) |>  
  fit(DataTrain)
```

We now have a fitted workflow that can be used to predict diamond prices.

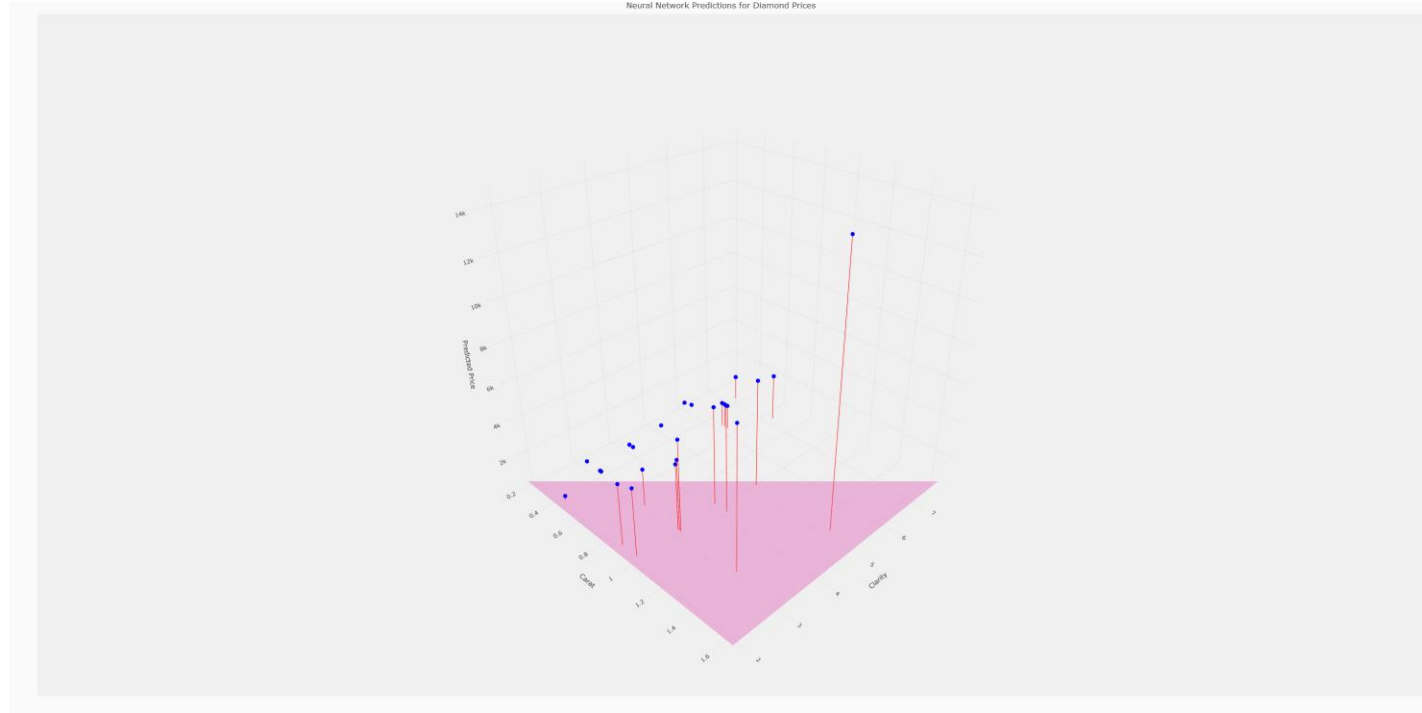
Buils a Simplified Neural Network in R

Generate predictions for the training dataset

```
DataTrainWithPredNN=augment(WFModel  
NN, new_data=DataTrain)  
metrics(DataTrainWithPredNN,  
truth=Price, estimate=.pred)
```

.metric	.estimator	.estimate
rmse	standard	197.886806
rsq	standard	0.995971
mae	standard	110.939037

Scatter Plot for the Diamonds Data



Buils a Simplified Neural Network in R

How well can the model predict new diamond prices?

```
DataTestWithPredNN=augment(WFModelN  
N, new_data=DataTest)  
metrics(DataTestWithPredNN,  
truth=Price, estimate=.pred)
```

.metric	.estimator	.estimate
rmse	standard	2201.6801187
rsq	standard	0.7087217
mae	standard	1272.8150033

Real World Example to Estimate Diamond Prices

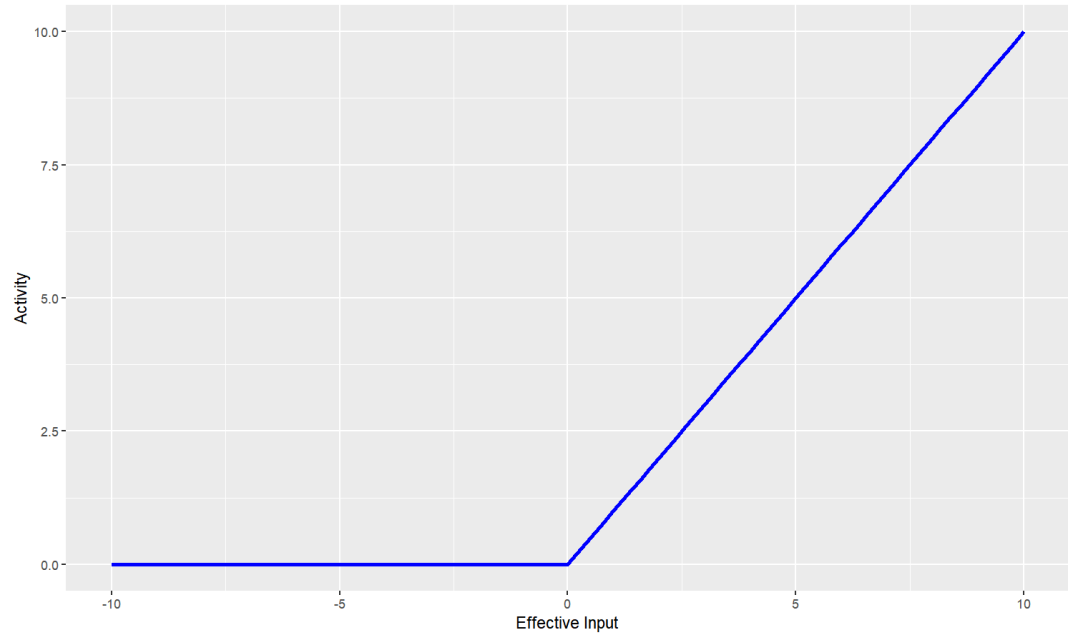
1. You will use all big *C* variables *Carat*, *Clarity*, *Cut*, and *Color*. *Cut* describes the quality of the cut of the diamond rated from 1 (lowest) to 6 (highest) and *Color* rates the color of a diamond from 1 (highest) to 7 (lowest)
2. Instead of using the `nnet` package, you will use the more advanced `brulee` package which is based on *PyTorch*, which is a Python library originally developed by *Facebook*.
3. We will tune the hyper-parameters of the neural network (e.g., the number of hidden units) using *cross validation*.

Major Differences: `nnet` and `brulee/PyTorch`

- `brulee` uses internally stop learning.
 - `epoch` setting refers to maximum *epochs*
 - from the *training* data set a *validation* set is held back.
 - when *validation error* stops decreasing for 5 epochs training is stopped.
- `brulee` allows to use **ReLU Activation Functions**
- Hyperparameters that can be tuned *using pytorch*
 - `epochs`: max number of epochs
 - `hidden_neurons`: number of hidden neurons and the number of hidden layers.
 - If you want to work with multiple hidden layers, you would have to assign a vector
`hidden_neurons=c(25, 30, 20)`
 - `dropout`: randomly switch off a predefined percentage of hidden neurons for each iteration of the optimizer process

ReLu Activation Function

$$Act_i = \max(0, I_i^{eff})$$



Two *ReLU* functions can be combined into one step function similar to *sigmoid* functions.

Why ReLU is better than Logistic

- ReLU has a constant slope
- Logistic function is always going to have a slope smaller than 1
 - becomes very small when effective inputs become larger
- Because of this, the Logistic function is ok for one-layer NNs, but is going to be problematic for many-layer NNs.
 - When the Optimizer adjusts individual *beta* values in a multi-layer Neural Network and calculates by how much they change from iteration to iteration, the weighted slopes of the activation functions in different layers are multiplied.
 - If these weighted slopes are smaller than one, each multiplication makes the change of the related *beta* smaller.
 - With enough layers, the calculated change might get close to zero.
- Consequently, the *beta* parameters will not change anymore, and the optimization process gets stuck.
- This problem is called the Vanishing Gradient

Now It's Time to Run the Real-World Analysis

Let's Run the Analysis

Download: `10-DeepLearnExerc100.Rmd`